CW1 #8.1: `LabeledCircle` Class

Create a subclass of `Circle` named `LabeledCircle` that adds simple metadata and behaviour, and overrides one method from `Circle`. You will **not** modify `Circle.java`; you will only write a new `LabeledCircle.java` file that compiles and works with the provided `Circle` implementation (from the Week 6 CW1).

Requirements

1. **Class Declaration**
   - `public class LabeledCircle extends Circle`

2. **State (new fields only)**
   - `private String label` — short text to identify the circle.
   - `private boolean filled` — whether the circle is visually filled (`true`) or just an outline (`false`).

   Keep centers and radius in the **superclass** (as they already exist in `Circle`).

3. **Constructors:** Provide two constructors that delegate appropriately to `Circle` via `super(...)`:
   - A **no-argument constructor** that uses super() to create a circle centred at the origin `(0, 0)` with radius `1.0`, then sets `label` to `"untitled"` and `filled` to `false`.
   - A **full constructor:** `public LabeledCircle(double centerX, double centerY, double radius, String label, boolean filled)`:
     - Must call `super(centerX, centerY, radius)`.
     - Sets the new fields accordingly (if `label` is null, store `"untitled"`).

4. **Getters & Setters (for new fields)**
   - `public String getLabel()`
   - `public boolean isFilled()`
   - `public void setLabel(String newLabel)` — if `newLabel` is `null`, store `"untitled"`.
   - `public void setFilled(boolean filled)`

5. **New Behaviours (added functionality):** Implement both of the following methods using the public methods of `Circle` (e.g., its getters and `setCenter`):
   - `public double getDiameter()`: Returns `2.0 * getRadius()`.
   - `public void moveBy(double dx, double dy)`: Translates the circle by `(dx, dy)`. Hint: use `getCenterX()`, `getCenterY()`, and `setCenter(...)` in `Circle`.

6. **Overriding:** Override `toString()` from `Circle`:
   - Return a string like: `LabeledCircle(center=(2.0, -1.5), radius=3.0, label="Room A", filled=true)`
   - **Hint:** You can add a double quotation mark character to a string like this: `String s = "\"";`

Notes:

- **Do not** copy or modify `Circle.java`. Your code must work with the given `Circle` class from the Week 6 CW1.
- Keep fields in `LabeledCircle` **private** and use getters/setters.
- Use `super(...)` in constructors to initialize the inherited state.
- Follow the output format closely in `toString()`.

Example Test Cases

```
// Example 1: default constructor + overridden toString
LabeledCircle a = new LabeledCircle();
System.out.println(a); // -> LabeledCircle(center=(0.0, 0.0), radius=1.0,
label="untitled", filled=false)
System.out.println(a.getDiameter());   // -> 2.0

// Example 2: full constructor + movement
LabeledCircle b = new LabeledCircle(2.0, -1.5, 3.0, "Room A", true);
System.out.println(b.getRadius());   // -> 3.0
System.out.println(b.isFilled());    // -> true
b.moveBy(-2.0, 1.5);                 // center becomes (0.0, 0.0)
System.out.println(b); // -> LabeledCircle(center=(0.0, 0.0), radius=3.0,
label="Room A", filled=true)

// Example 3: polymorphism with overridden method
Circle ref = new LabeledCircle(1.0, 1.0, 2.0, "C2", false);
System.out.println(ref.getArea()); // uses Circle.getArea()
System.out.println(ref);      // calls LabeledCircle.toString() at runtime
```

Submission guidance

- Submit only your `LabeledCircle.java` file.
- Include all constructors, getters, setters, and methods exactly as listed.